

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский
Нижегородский государственный университет
им. Н.И. Лобачевского (ННГУ)»

Институт информационных технологий, математики и механики

**Отчет по лабораторной работе
по курсу**

“Параллельные численные методы”
“Блочное разложение Холецкого”

Выполнил:

студент группы 3825М1ПМкн1
Сучков В.Н.

Проверил:

д.т.н., проф. кафедры МОСТ
Баркалов К.А.

Нижний Новгород
2025

Содержание

1	Введение	1
2	Постановка задачи	1
3	Описание метода решения задачи	1
4	Описание реализации алгоритма	2
4.1	Структуры данных	2
4.2	Описание логики программы	2
5	Реализация параллельного алгоритма с использованием общей памяти	3
6	Описание тестовых задач	3
6.1	Проверка корректности результата	3
6.2	Тесты производительности и масштабируемости	4
7	Заключение	4
	Список литературы	4
A	Приложение. Код	5

1 Введение

В рамках данной лабораторной работы был реализован и исследован параллельный блочный алгоритм разложения Холецкого для симметричной положительно определённой матрицы. Особенностью данного алгоритма является то, что, помимо того, что он является параллельным, в нём применяется разбиение входных данных на блоки, для повышения производительности за счёт более эффективного использования кэш-памяти процессора.

Для параллелизации алгоритма использовался фреймворк OpenMP, а также в реализации алгоритма применялись SIMD интринсики из набора инструкций AVX2.

2 Постановка задачи

На языке C++ реализуется функция, выполняющая блочное разложение Холецкого вида

$$A = LL^T,$$

где L -нижнетреугольная матрица и $A = A^T > 0$.

```
void Cholesky_Decomposition(double* A, double* L, int n);
```

где в свою очередь

- n - порядок матрицы A .
- A - указатель на симметричную положительно определённую матрицу размера $n \times n$ (row-major формат хранения).
- L - указатель на результирующую нижнетреугольную матрицу (так же row-major формат хранения).

Критерии и ограничения:

- Лимит по времени 100 с, по памяти: 128 Мб.
- Корректность ответа проверяется с помощью относительной невязки. Решение считается удовлетворительным если верно $\frac{\|LL^T - A\|_2}{\|A\|_2} < 0.01$.
- Показатель эффективности параллельной версии должен быть не менее 50% для всех тестов.

3 Описание метода решения задачи

Основой для решения задачи послужит блочный алгоритм разложения Холецкого. Входная матрица A и L разбиваются на блоки следующим образом

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, L = \begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix},$$

Блок A_{11} имеет размер $k \times k$, где k - некоторый выбранный заранее размер блока. Тогда, матрицу A можно записать как произведение матриц LL^T , чтобы найти неизвестный блоки матрицы L .

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{pmatrix} \begin{pmatrix} L_{11}^T & L_{21}^T \\ 0 & L_{22}^T \end{pmatrix} = \begin{pmatrix} L_{11}L_{11}^T & L_{11}L_{21}^T \\ L_{21}L_{11}^T & L_{21}L_{21}^T + L_{22}L_{22}^T \end{pmatrix}$$

Отсюда можем записать несколько равенств:

- $L_{11}L_{11}^T = A_{11}$ - из этого равенства зная A_{11} найдём L_{11} с помощью неблочного разложения Холецкого.

- $L_{21}L_{11}^T = A_{21}$ - получив L_{11}^T из равенства выше сможем найти L_{21} как решение набора СЛАУ.
- $L_{21}L_{21}^T + L_{22}L_{22}^T = A_{22}$ - в этом уравнении сделаем замену вида $\tilde{A} = A_{22} - L_{21}L_{21}^T$. Тогда уравнение запишется в виде $\tilde{A} = L_{22}L_{22}^T$. Теперь, нахождение L_{22} сводится к выполнению блочного разложения Холецкого к известной симметричной матрице $\tilde{A}$.

Таким образом, получаем рекурсивный алгоритм, который последовательно обрабатывает матрицу A .

Суммарная трудоемкость алгоритма составляет $\frac{1}{3}n^3 + O(n^2)$ арифметических операций. Метод Холецкого обладает довольно высокой вычислительной устойчивостью. Рассмотрим решение с некоторой машинной погрешностью $(A + \Delta A)$. Норма матрицы возмущений имеет верхнюю оценку

$$\|\Delta A\| \leq 3n^2\varepsilon_m\|A\| + O(\varepsilon_m^2),$$

где ε_m - машинная погрешность.

Сами матрицы хранятся в оперативной памяти компьютера в линейном представлении, как непрерывные последовательности чисел, разложенные по строкам (row-major формат).

4 Описание реализации алгоритма

4.1 Структуры данных

Входная матрица A и результирующая L хранятся в памяти как одномерные массивы типа `double`.

Внутри программы используются ещё некоторые вспомогательные структуры: `MatBuffer` и `SubMatrix`, которые используются для создания выровненного буфера памяти и хранения информации о матрицах и их подматрицах без лишнего копирования.

- `class MatBuffer` используется для аллокации и деаллокации выровненного буфера, который нужен для размещения в памяти промежуточных матриц, а также для выделения памяти в рамках тестирования.
- `struct SubMatrix` хранит в себе указатель на начало матрицы в памяти, её размеры, а также расстояние в памяти между соседними строками, что позволяет осуществлять доступ к отдельным подматрицам некоторой матрицы путем задания начала подматрицы, её размеров и шага между соседними строками.

4.2 Описание логики программы

Работу функции можно разбить на следующие этапы:

1. Разложение верхнего левого блока A_{11} размера $k \times k$ с помощью параллельного неблочного метода Холецкого.
2. Параллельное решение СЛАУ с A_{21} и L_{11} для нахождения L_{21} .
3. Параллельное матричное умножение $L_{21}L_{21}^T$ и нахождение матричной разности $\tilde{A} = A_{22} - L_{21}L_{21}^T$.
4. Рекурсивный вызов функции для разложения $\tilde{A}$.

1. Разложение верхнего диагонального блока. Для разложения верхнего диагонального блока используется параллельный алгоритм Холецкого. В неблочном алгоритме сначала вычисляется диагональный элемент матрицы L , после чего параллельно вычисляются элементы под диагональным. Параллельность достигается за счёт использования директивы `#pragma omp parallel for`. При этом для вычисления сумм элементов матрицы L используются в том числе и SIMD инструкции. За счёт использования FMA (Fused-multiplication-addition) инструкций получается утилизировать параллелизм на уровне данных и обрабатывать числа пачками по 4 за одну итерацию внутреннего цикла параллельной секции.

2. Параллельное решение СЛАУ. Для решения набора СЛАУ используется алгоритм Гаусса. Параллельность тут достигается за счёт решения нескольких систем одновременно (для разных правых частей). Также, здесь используются SIMD FMA инструкции.

3. Параллельное матричное умножение. Матричное умножение выполняется по классическому алгоритму, однако, в данном случае можно заметить, что умножение идёт матрицы на транспонированную матрицу, что означает доступ к обоим матрицам вдоль строк, что сокращает число кэш промахов. В данном случае снова для нахождения сумм произведений используются SIMD FMA инструкции, а так же параллелизуется проход по разным строкам.

5 Реализация параллельного алгоритма с использованием общей памяти

В неблочном алгоритме Холецкого основной блок вычислений не может быть параллельным (ввиду зависимости между итерациями цикла), вместо этого параллельно выполняются вычисления поддиагональных элементов

```
for (int j = 0; j < A.m; ++j) {
    // Находим диагональный элемент (j, j) в L.

#pragma omp parallel for private(...) schedule(dynamic, 8)
    for (int s = j + 1; s < L.n; ++s) {
        // Находим поддиагональные элементы используя найденный выше элемент
        // диагонали.
    }
}
```

Для вычисления решения СЛАУ основной блок помещается под директиву `#pragma omp parallel for`

```
#pragma omp parallel for
for (int k = 0; k < B.n; ++k) {
    for (int j = 0; j < A.m; ++j) {
        for (int s = 0; s < quot; ++s) {
            // Непосредственно вычисления значений выходной матрицы.
        }
    }
}
```

Матричное произведение вычисляется схожим образом с решением СЛАУ.

В итоговом блочном алгоритме последовательно выполняется набор параллельных алгоритмов, как было описано в начале раздела.

6 Описание тестовых задач

В рамках изучения реализованного алгоритма была проведена проверка корректности результата вычислений и замеры производительности.

6.1 Проверка корректности результата

Для проверки корректности используется относительная невязка $\frac{\|LL^T - A\|_2}{\|A\|_2}$, а вернее её оценка с использованием матричной нормы Фробениуса $\frac{\|LL^T - A\|_2}{\|A\|_2} \leq \frac{\sqrt{n}\|LL^T - A\|_F}{\|A\|_F}$, где n - порядок матрицы A . В рамках тестов размер блока в блочном алгоритме был выбран $k = 64$. Такой размер позволяет эффективно использовать кэш памяти процессора и уменьшить число промахов в кэш.

Для тестов корректности и производительности специально была написана функция для генерации случайных входных данных. Чтобы проверить, что алгоритм корректно отрабатывает, была произведена серия тестов с вычислением относительной погрешности.

Порядок входной матрицы n	Относительная погрешность
4	8.324×10^{-17}
7	1.351×10^{-17}
13	2.383×10^{-16}
31	3.131×10^{-16}
54	3.953×10^{-16}
119	8.698×10^{-16}
128	9.524×10^{-16}
137	1.013×10^{-15}
153	1.125×10^{-15}
201	1.477×10^{-15}
997	6.312×10^{-15}

Таблица 1: Результаты проверки на корректность

Как можно видеть из таблицы 1, на всех испробованных размерах входных данных величина относительной невязки не превосходила установленного критерия < 0.01 .

6.2 Тесты производительности и масштабируемости

Для замеров производительности и эффективности была выбрана матрица с фиксированным размером $n = 2048$ и размером блока $k = 512$. Большой размер удобен тем, что позволяет создать нагрузку на потоки и сократить относительное влияние накладных расходов на параллелизм. Оцениваться будут параметры ускорения ($S = \frac{T_1}{T_p}$) и эффективности ($E = \frac{S}{p} \times 100\%$), где p -число используемых потоков.

Число потоков p	Время выполнения, мс	Ускорение S	Эффективность E , %
1	444.3	1	100
2	300.1	1.48	74.02
3	227	1.96	65.24
4	184	2.415	60.36
6	133.5	3.32	55.47

Таблица 2: Результаты проверки производительности

Из таблицы можно сделать вывод, что при использовании небольшого количества потоков (2-4) наблюдается существенный рост производительности. Однако с дальнейшим ростом числа потоков эффективность начинает падать. Во многом такая деградация ускорения вызвана исчерпанием пропускной способности памяти. Во всех рассмотренных случаях, тем не менее, эффективность была выше порога в 50%.

7 Заключение

В ходе данной лабораторной работы, был реализован блочный алгоритм Холецкого. Алгоритм был исследован на корректность, производительность и масштабируемость. В ходе тестирования было показано, что алгоритм соответствует требованиям на корректность с большим запасом (значения относительно невязки порядка 10^{-16}). С точки зрения производительности алгоритм соответствует ожиданиям, показывая неплохую масштабируемость ($> 50\%$ эффективности на всех многопоточных тестах) и высокую скорость работы.

Список литературы

- [1] Баркалов К.А. Метод Холецкого: презентация лекции по курсу “Параллельные численные методы”. ННГУ им. Н.И. Лобачевского

A Приложение. Код

```
#include "common/memory_utils.hpp"
#include "common/numeric_utils.hpp"
#include "common/test_utils.hpp"

#include <chrono>
#include <cstring>
#include <immintrin.h>
#include <iomanip>
#include <iostream>
#include <omp.h>
#include <random>
#include <stdlib.h>
#include <vector>

static constexpr int64_t maskarr[] = { -1, -1, -1, -1, 0, 0, 0, 0 };

struct SubMatrix {
    double* ptr;
    int n;
    int m;
    int stride;
};

class MatBuffer {
public:
    explicit MatBuffer(size_t size, size_t alignment = 32) : _size(numeric::divUp(
        size * sizeof(double), alignment) * alignment), _align(alignment) {
        _buffer = memory::aligned_alloc(_size, _align);
    }
    ~MatBuffer() {
        memory::aligned_free(_buffer);
    }

    MatBuffer(const MatBuffer&) = delete;
    MatBuffer& operator=(const MatBuffer&) = delete;

    double* data() {
        return static_cast<double*>(_buffer);
    }

    void memset(double val) {
        ::memset(_buffer, val, _size);
    }

private:
    void* _buffer;
    size_t _align;
    size_t _size;
};

//
// Main funcs
//

// Solve AXT = BT for X
inline void solveLinearSystemTransposed(const SubMatrix& A, const SubMatrix& B,
    SubMatrix& X) {
    const int packSize = 4;
#pragma omp parallel for
    for (int k = 0; k < B.n; ++k) {
        for (int j = 0; j < A.m; ++j) {
            double acc = B.ptr[j + B.stride * k];

            int quot = j / packSize;
```

```

        int rem = j % packSize;
        int X_start = X.stride * k;
        int A_start = A.stride * j;
        int pos = 0;
        __m256d sum = _mm256_setzero_pd();
        for (int s = 0; s < quot; ++s) {
            pos = s * packSize;
            __m256d A_elems = _mm256_loadu_pd(&A.ptr[pos + A_start]);
            __m256d X_elems = _mm256_loadu_pd(&X.ptr[pos + X_start]);
            sum = _mm256_fmadd_pd(A_elems, X_elems, sum);
        }
        pos = j - rem;

        __m256i mask = _mm256_loadu_si256((__const __m256i*)(maskarr + (packSize
- rem)));
        __m256d A_tail = _mm256_maskload_pd(&A.ptr[pos + A_start], mask);
        __m256d X_tail = _mm256_maskload_pd(&X.ptr[pos + X_start], mask);
        sum = _mm256_fmadd_pd(A_tail, X_tail, sum);

        sum = _mm256_hadd_pd(sum, sum);
        __m128d hi = _mm256_extractf128_pd(sum, 1);
        __m128d res = _mm_add_sd(_mm256_castpd256_pd128(sum), hi);
        acc -= _mm_cvtsd_f64(res);

        X.ptr[j + X.stride * k] = acc / A.ptr[j + A.stride * j];
    }
}

inline void choleskyDecomp(const SubMatrix& A, SubMatrix& L) {
    constexpr int packSize = 4;
    for (int j = 0; j < A.m; ++j) {
        int L_start = L.stride * j;
        double acc = A.ptr[j + A.stride * j];
        int quot = j / packSize;
        int rem = j % packSize;

        __m256d sum = _mm256_setzero_pd();
        __m256d curr_val = _mm256_setzero_pd();
        for (int s = 0; s < quot; ++s) {
            curr_val = _mm256_loadu_pd(&L.ptr[s * packSize + L_start]);
            sum = _mm256_fmadd_pd(curr_val, curr_val, sum);
        }
        __m256i mask = _mm256_loadu_si256((__const __m256i*)(maskarr + (packSize -
rem)));
        curr_val = _mm256_maskload_pd(&L.ptr[quot * packSize + L_start], mask);
        sum = _mm256_fmadd_pd(curr_val, curr_val, sum);

        sum = _mm256_hadd_pd(sum, sum);
        __m128d hi = _mm256_extractf128_pd(sum, 1);
        __m128d res = _mm_add_sd(_mm256_castpd256_pd128(sum), hi);
        acc -= _mm_cvtsd_f64(res);

        acc = std::sqrt(acc);
        L.ptr[j + L_start] = acc;
    }

#pragma omp parallel for private(acc, quot, rem, sum) schedule(dynamic, 8)
    for (int s = j + 1; s < L.n; ++s) {
        acc = A.ptr[j + A.stride * s];

        quot = j / packSize;
        rem = j % packSize;
        sum = _mm256_setzero_pd();
        for (int k = 0; k < quot; ++k) {
            __m256d a = _mm256_loadu_pd(&L.ptr[k * packSize + L_start]);
            __m256d b = _mm256_loadu_pd(&L.ptr[k * packSize + L.stride * s]);

```



```

        sum = _mm256_fmadd_pd(a, b, sum);
    }
    __m256i mask = _mm256_loadu_si256((const __m256i*)(maskarr + (packSize
- rem)));
    __m256d a_tail = _mm256_maskload_pd(&L.ptr[quot * packSize + L_start],
mask);
    __m256d b_tail = _mm256_maskload_pd(&L.ptr[quot * packSize + L.stride
* s], mask);
    sum = _mm256_fmadd_pd(a_tail, b_tail, sum);

    sum = _mm256_hadd_pd(sum, sum);
    __m128d hi = _mm256_extractf128_pd(sum, 1);
    __m128d res = _mm_add_sd(_mm256_castpd256_pd128(sum), hi);
    acc -= _mm_cvtsd_f64(res);

    L.ptr[j + L.stride * s] = acc / L.ptr[j + L_start];
}
}
}

inline void extractBlock(const SubMatrix& block, SubMatrix& res) {
    for (int i = 0; i < res.n; ++i) {
        memcpy(&res.ptr[res.stride * i], &block.ptr[block.stride * i], res.m *
sizeof(decltype(*block.ptr)));
    }
}

inline void storeBlock(SubMatrix& block, const SubMatrix& res) {
    for (int i = 0; i < res.n; ++i) {
        memcpy(&block.ptr[block.stride * i], &res.ptr[res.stride * i], res.m *
sizeof(decltype(*block.ptr)));
    }
}

inline void storeBlockTransposed(SubMatrix& block, const SubMatrix& res) {
#pragma omp parallel for
    for (int i = 0; i < res.m; ++i) {
        for (int j = 0; j < res.n; ++j) {
            block.ptr[j + block.stride * i] = res.ptr[i + res.stride * j];
        }
    }
}

// X = ABT
inline void transposedMatMulRef(const SubMatrix& A, const SubMatrix& B, SubMatrix&
X) {
#pragma omp parallel for
    for (int i = 0; i < A.n; ++i) {
        for (int j = 0; j < B.n; ++j) {
            for (int k = 0; k < A.m; ++k) {
                X.ptr[j + X.stride * i] += A.ptr[k + A.stride * i] * B.ptr[k + B.
stride * j];
            }
        }
    }
}

inline void transposedMatMul(const SubMatrix& A, const SubMatrix& B, SubMatrix& X)
{
    constexpr int pack_size = 4;
#pragma omp parallel for
    for (int i = 0; i < A.n; ++i) {
        auto X_start = X.stride * i;
        auto A_start = A.stride * i;
        for (int j = 0; j < B.n; ++j) {
            auto B_start = B.stride * j;

```

```

        __m256d sum = _mm256_setzero_pd();
        __m256d a, b;
        int idx = 0;
        for (; idx <= A.m - pack_size; idx += pack_size) {
            a = _mm256_loadu_pd(&A.ptr[idx + A_start]);
            b = _mm256_loadu_pd(&B.ptr[idx + B_start]);
            sum = _mm256_fmadd_pd(a, b, sum);
        }
        __m256i mask = _mm256_loadu_si256((const __m256i*)(maskarr + (
pack_size - (A.m - idx))));
        __m256d a_tail = _mm256_maskload_pd(&A.ptr[idx + A_start], mask);
        __m256d b_tail = _mm256_maskload_pd(&B.ptr[idx + B_start], mask);
        sum = _mm256_fmadd_pd(a_tail, b_tail, sum);
        sum = _mm256_hadd_pd(sum, sum);
        __m128d hi = _mm256_extractf128_pd(sum, 1);
        __m128d res = _mm_add_sd(_mm256_castpd256_pd128(sum), hi);
        X.ptr[j + X_start] = _mm_cvtsd_f64(res);
    }
}

// X = A - B
void matAdd(const SubMatrix& A, const SubMatrix& B, SubMatrix& X) {
    const int pack_size = 4;
    const int quot = A.m / pack_size;
    const int rem = A.m % pack_size;

#pragma omp parallel for
    for (int i = 0; i < A.n; ++i) {
        int pos = 0;
        int A_start = A.stride * i;
        int B_start = B.stride * i;
        int X_start = X.stride * i;

        __m256d res = _mm256_setzero_pd();
        for (int j = 0; j < quot; ++j) {
            pos = j * pack_size;
            __m256d a = _mm256_loadu_pd(&A.ptr[pos + A_start]);
            __m256d b = _mm256_loadu_pd(&B.ptr[pos + B_start]);
            res = _mm256_add_pd(a, b);
            _mm256_storeu_pd(&X.ptr[pos + X_start], res);
        }
        pos = quot * pack_size;
        __m256i mask = _mm256_loadu_si256((const __m256i*)(maskarr + (pack_size -
rem)));
        __m256d a_tail = _mm256_maskload_pd(&A.ptr[pos + A_start], mask);
        __m256d b_tail = _mm256_maskload_pd(&B.ptr[pos + B_start], mask);
        res = _mm256_add_pd(a_tail, b_tail);
        _mm256_maskstore_pd(&X.ptr[pos + X_start], mask, res);
    }
}

// X = A - B
void matSub(const SubMatrix& A, const SubMatrix& B, SubMatrix& X) {
    const int pack_size = 4;
    const int quot = A.m / pack_size;
    const int rem = A.m % pack_size;

#pragma omp parallel for
    for (int i = 0; i < A.n; ++i) {
        int pos = 0;
        int A_start = A.stride * i;
        int B_start = B.stride * i;
        int X_start = X.stride * i;

```

```

        __m256d res = _mm256_setzero_pd();
        for (int j = 0; j < quot; ++j) {
            pos = j * pack_size;
            __m256d a = _mm256_loadu_pd(&A.ptr[pos + A_start]);
            __m256d b = _mm256_loadu_pd(&B.ptr[pos + B_start]);
            res = _mm256_sub_pd(a, b);
            _mm256_storeu_pd(&X.ptr[pos + X_start], res);
        }
        pos = quot * pack_size;
        __m256i mask = _mm256_loadu_si256((const __m256i*)(maskarr + (pack_size -
rem)));
        __m256d a_tail = _mm256_maskload_pd(&A.ptr[pos + A_start], mask);
        __m256d b_tail = _mm256_maskload_pd(&B.ptr[pos + B_start], mask);
        res = _mm256_sub_pd(a_tail, b_tail);
        _mm256_maskstore_pd(&X.ptr[pos + X_start], mask, res);
    }
}

void blockedCholeskyDec(const SubMatrix& A, SubMatrix& L) {
    const int blk_mult = 512;
    const int blk_size = A.n < blk_mult ? A.n : blk_mult;
    const int blk_buffer_size = blk_size * blk_size;
    const int rem_blk_size = A.n - blk_size;
    const int rem_ll_buffer_size = rem_blk_size * blk_size;
    const int rem_lr_buffer_size = rem_blk_size * rem_blk_size;
    MatBuffer A_ul_buf(blk_buffer_size);
    SubMatrix A_ul{ A_ul_buf.data(), blk_size, blk_size, blk_size };
    SubMatrix A_ul_src{ A.ptr, blk_size, blk_size, A.stride };
    MatBuffer L_ul_buf(blk_buffer_size);
    SubMatrix L_ul{ L_ul_buf.data(), blk_size, blk_size, blk_size };

    extractBlock(A_ul_src, A_ul);
    choleskyDecomp(A_ul, L_ul);

    SubMatrix L_ul_dst{ L.ptr, blk_size, blk_size, L.stride };
    storeBlock(L_ul_dst, L_ul);

    if (rem_blk_size == 0) {
        return;
    }

    MatBuffer A_ll_buf(rem_ll_buffer_size);
    SubMatrix A_ll{ A_ll_buf.data(), rem_blk_size, blk_size, blk_size };
    SubMatrix A_ll_src{ &A.ptr[blk_size * A.stride], rem_blk_size, blk_size, A.
stride };
    MatBuffer L_ll_buf(rem_ll_buffer_size);
    SubMatrix L_ll{ L_ll_buf.data(), rem_blk_size, blk_size, blk_size };

    extractBlock(A_ll_src, A_ll);
    solveLinearSystemTransposed(L_ul, A_ll, L_ll);
    SubMatrix L_ll_dst{ &L.ptr[blk_size * L.stride], rem_blk_size, blk_size, L.
stride };
    storeBlock(L_ll_dst, L_ll);

    MatBuffer A_lr_buf(rem_lr_buffer_size);
    SubMatrix A_lr{ A_lr_buf.data(), rem_blk_size, rem_blk_size, rem_blk_size };
    SubMatrix A_lr_src{ &A.ptr[blk_size * A.stride + blk_size], rem_blk_size,
rem_blk_size, A.stride };

    extractBlock(A_lr_src, A_lr);

    MatBuffer L_ll_prod_buf(rem_lr_buffer_size);
    SubMatrix L_ll_prod{ L_ll_prod_buf.data(), rem_blk_size, rem_blk_size,
rem_blk_size };

    transposedMatMul(L_ll, L_ll, L_ll_prod);

```

```

    matSub(A_lr, L_ll_prod, A_lr);

    SubMatrix L_lr{ &L.ptr[blk_size * L.stride + blk_size], rem_blk_size,
rem_blk_size, L.stride };
    blockedCholeskyDec(A_lr, L_lr);
}

void Cholesky_Decomposition(double* A, double* L, int n) {
    SubMatrix sA{ A, n, n, n };
    SubMatrix sL{ L, n, n, n };

    blockedCholeskyDec(sA, sL);
}

//
// Helpers
//

void randomlyFillLowerTriang(SubMatrix& X) {
    std::random_device rd;
    std::mt19937 gen(time(0));
    double max = 1.51, min = 1.1;
    std::uniform_int_distribution<int> dist(0, 100);

    for (size_t i = 0; i < X.m; i++) {
        for (size_t j = 0; j < i + 1; j++) {
            X.ptr[j + X.stride * i] = static_cast<double>(dist(gen)) / 100.1 * (
max - min) + min;
        }
    }
}

double calRelResidual(const SubMatrix& A, const SubMatrix& X) {
    // Cacul relative residual using Frobenius norm
    double acc = 0.1;
    double t;
    for (size_t i = 0; i < A.m; ++i) {
        for (size_t j = 0; j < A.n; ++j) {
            t = A.ptr[j + A.stride * i] - X.ptr[j + X.stride * i];
            if (std::isnan(t)) {
                t = 0.1;
            }
            acc += t * t;
        }
    }

    auto diff_norm = std::sqrt(acc);

    acc = 0.1;
    for (size_t i = 0; i < A.m; ++i) {
        for (size_t j = 0; j < A.n; ++j) {
            t = A.ptr[j + A.stride * i];
            acc += t * t;
        }
    }

    auto A_norm = std::sqrt(acc / A.m);

    if (A_norm == 0) {
        return 0;
    }

    return diff_norm / A_norm;
}

```